

AeroPulse India — Real-Time AQI & Telemetry Predictor

Course: Mathematics for Intelligent Systems 3 (23MAT204)

Group: B16

Members:

Adithya.R - CB.AI.U4AID25102

S. Manoj - CB.AI.U4AID152

Introduction

AeroPulse India is a full-stack environmental command center that processes live atmospheric sensor data and forecasts short-term air quality trends across major Indian cities.

- **Core Function:** Ingests real-time pollutant feeds (PM_{2.5}, PM₁₀, NO₂, SO₂, O₃, CO) via the Open Weather API and projects concentration levels.
- **Live & 2-Hour Historical Ingestion:** Ingest real-time and past 2-hour telemetry feeds across Indian cities via the OpenWeather Air Pollution API.
- **System Architecture:** Built with a Python/Flask microservice backend using NumPy for high-speed matrix inversion, paired with an interactive dark-mode dashboard featuring Leaflet.js geospatial mapping and dynamic Chart.js predictive trendlines.

Literature Survey

Title and Authors	Methodology & Algorithms	Key Achievements
<i>Air Quality Index Forecasting Using Time Series Models</i> (Box & Jenkins et al.)	Autoregressive Integrated Moving Average (ARIMA) statistical modeling.	Effective at identifying linear historical patterns and short-term stationary trends.
<i>Deep Learning for Urban Air Pollution Prediction</i> (Wang et al.)	Long Short-Term Memory (LSTM) Neural Networks and Recurrent Neural Networks (RNNs).	High accuracy in capturing complex, non-linear long-term temporal dependencies across high-dimensional datasets.
<i>Geospatial Sensing and Pollution Interpolation</i> (Zhang & Liu)	Kriging Interpolation & Random Forest Regression on static sensor networks.	Provides spatial mapping of pollutant distribution across unmonitored geographical regions.

Problem Statement

To build **AeroPulse India**, an interactive environmental dashboard that ingests live and 2-hour historical atmospheric sensor feeds via the OpenWeather API to display individual pollutant molecule concentrations dynamically rank cities by AQI severity, and visualize short-term air quality trends across urban centers.

Objectives

- **Live Data Ingestion Pipeline:** Establish an automated microservice layer using Python and Flask to fetch real-time air quality telemetry data via the OpenWeather API across major Indian cities.
- **Interactive Visualization Dashboard:** Develop a frontend interface featuring Leaflet.js geospatial mapping, dynamic Chart.js predictive trendlines, and pollutant break-down indicators.
- **Pollutant Molecule Breakdown:** Measure, display, and analyze the individual concentration levels ($\mu\text{g}/\text{m}^3$) of key gaseous molecules (PM_{2.5}, PM₁₀, NO₂, SO₂, O₃, CO) against safety limits.
- **City AQI Ranking System:** Generate a dynamic rank list ordering cities in real time based on their calculated Air Quality Index (AQI) severity.

Methodology

1. **Data Acquisition & Ingestion:** Ingest live and 2-hour historical atmospheric sensor feeds across major Indian cities via the OpenWeather Air Pollution API using Python `requests`.
2. **Backend Processing & AQI Computation:** Parse raw JSON telemetry in a Flask backend to extract concentration values for key pollutants and calculate overall AQI sub-indices.
3. **City Ranking Engine:** Execute a dynamic sorting algorithm on calculated AQI values to automatically rank monitored cities from most polluted to cleanest.
4. **Fallback Cache Handling:** Store baseline data in memory to serve local telemetry seamlessly if API rate limits or network downtime occur.
5. **Geospatial & Trend UI Rendering:** Transmit processed payloads to the frontend to plot map pins via Leaflet.js, display individual gas concentration metrics, and render 2-hour trendlines using Chart.js.

Future Work

- **Integrate Weather Data (OpenWeather Weather API):** Fetch temperature, humidity, and wind speed alongside pollution data to correlate weather conditions with AQI spikes.
- **Extended Historical Window:** Expand from a 2-hour window to a 24-hour historical log to display full-day pollution trendlines.
- **Expand City Coverage:** Integrate additional major cities and regional towns across India into the OpenWeather API fetching pipeline to provide broader geographic coverage and more comprehensive nationwide rankings.

Conclusion

- **Real-Time Data Pipeline:** Successfully built a web platform that ingests live and 2-hour historical atmospheric telemetry across Indian cities via the OpenWeather API.
- **Multi-Gas Analysis:** Accurately displays individual concentration levels alongside composite AQI ratings.
- **Dynamic Ranking:** Automatically updates city rankings from most polluted to cleanest based on live pollution severity.
- **Geospatial & Visual Analytics:** Features an interactive Leaflet.js map for spatial tracking and Chart.js trendlines for historical data visualization.
- **High Availability Design:** Includes an in-memory baseline fallback mechanism to ensure seamless performance during API downtime or network latency.

References

- OpenWeather API Documentation: OpenWeather Ltd., Air Pollution API — Current, Historical, and Forecast Ambient Air Quality Data, 2026. [Online]. Available: <https://openweathermap.org/api/air-pollution>
- Central Pollution Control Board (CPCB): Ministry of Environment, Forest and Climate Change, Government of India, National Ambient Air Quality Standards (NAAQS) and AQI Sub-index Breakdowns, 2014. [Online]. Available: <https://cpcb.nic.in>
- Leaflet Documentation: V. Agafonkin, Leaflet: An Open-Source JavaScript Library for Interactive Mobile-Friendly Maps, 2024. [Online]. Available: <https://leafletjs.com>
- Chart.js Documentation: N. Downie and Chart.js Contributors, Chart.js: Simple yet Flexible JavaScript Charting for Designers & Developers, 2025. [Online]. Available: <https://www.chartjs.org>
- Flask Documentation: Pallets Projects, Flask: A Lightweight WSGI Web Application Framework in Python, 2024. [Online]. Available: <https://flask.palletsprojects.com>